

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf_b9d6fa25-e41\agent-ac55f55138fe9494a.jsonl`
- SHA-256: `f954fc5e0f6615352fd3cb8b1e7e3fbd0449f7cd8d3d0bf015ceb82f0bfac16d`
- Source modified: `2026-06-25T17:39:20+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_b9d6fa25-e41`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`
```

Transcript

1. user (2026-06-25T17:38:27.213Z)

Adversarially verify this PR-review finding for PR #385 (branch b1-p9/remove-fabricated-motion-data checked out; read the code ? STRICT READ-ONLY, do NOT edit any file). Be skeptical ? default to INVALID if wrong, already-handled, or a non-issue.

Finding [NIT] CSV export renders the literal string 'None' for motion shot_count instead of blank

Location: backend/tools/export.py:146

Detail: segments_to_csv does `shot = md.get("shot\_count", "")`. Because the key is now present with value None, the default "" never applies and the CSV cell renders the literal string `None` (not empty). Cosmetic only ? no crash, and format_rally_summary (export.py:73-74) and the worker intervals.csv (worker/__main__.py:83, reads a flat dict key that is absent for motion) are unaffected. Contrast: the rally-summary table correctly does `shot = ...get("shot\_count"); "-" if shot is None else str(shot)`.

Suggestion: For consistency with the '-'/blank convention, coerce None to blank: `shot = md.get("shot\_count") or ""` (or `"" if md.get("shot\_count") is None else md["shot\_count"]`). Optional polish, not blocking.

Return isReal + adjustedSeverity (INVALID if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T17:38:29.119Z)

I'll verify this finding by reading the actual code at the cited locations.

3. user (2026-06-25T17:38:30.328Z)

```
60         )
61         header = (
62             f"=== {title}: {len(segs)} rallies, "
63             f"{_fmt_hms(total_dur)} ({total_dur:.1f}s) total play ==="
64         )
65         if not segs:
66             return header + "\n (no rallies detected)"
67
68         lines = [header, f"{'#':>3} {'start':<8} {'end':<8} {'dur(s)':>7} {'shots':>6}"]
69         for i, s in enumerate(segs, start=1):
70             start = float(s.get("start_time", 0.0))
71             end = float(s.get("end_time", start))
72             dur = float(s.get("duration", end - start))
73             shot = _segment_metadata(s).get("shot_count")
74             shot_str = "-" if shot is None else str(shot)
75             lines.append(
76                 f"{i:>3} {_fmt_hms(start):<8} {_fmt_hms(end):<8} {dur:>7.1f}
{shot_str:>6}"
```

```

77         )
78         return "\n".join(lines)
79
80
81     def basic_stats(segments: Iterable[Dict[str, Any]]) -> Dict[str, Any]:
82         """Return simple aggregate stats for a list of play segments.
83
84         Keys: count, total_duration_s, avg_duration_s, min_duration_s, max_duration_s.
85         All durations are in seconds (float).
86         """
87         segs = _to_list(segments)
88         if not segs:
89             return {
90                 "count": 0,
91                 "total_duration_s": 0.0,
92                 "avg_duration_s": 0.0,
93                 "min_duration_s": 0.0,
94                 "max_duration_s": 0.0,
95             }
96         durs = [float(s.get("duration", (s.get("end_time", 0) - s.get("start_time", 0))))
97 for s in segs]
98         total = sum(durs)
99         return {
100             "count": len(segs),
101             "total_duration_s": round(total, 2),
102             "avg_duration_s": round(total / len(durs), 2),
103             "min_duration_s": round(min(durs), 2),
104             "max_duration_s": round(max(durs), 2),
105         }
106
107     def segments_to_json(
108         segments: Iterable[Dict[str, Any]],
109         video_id: str,
110         video_path: Optional[str] = None,
111         meta: Optional[Dict[str, Any]] = None,
112     ) -> Dict[str, Any]:
113         """Return a JSON-serializable timeline dict.
114
115         Shape (stable):
116         {
117             "video_id": "...",
118             "video_path": "...",
119             "meta": {...},
120             "segments": [ {"id":, "start_time":, "end_time":, "duration":, "metadata": {...}},
... ],
121             "stats": basic_stats(...)
122         }
123         """
124         segs = _to_list(segments)
125         return {
126             "video_id": video_id,
127             "video_path": video_path,
128             "meta": meta or {},
129             "segments": segs,
130             "stats": basic_stats(segs),
131         }
132
133
134     def segments_to_csv(segments: Iterable[Dict[str, Any]], include_header: bool = True) ->
str:
135         """Return a CSV string (start,end,duration,shot_count,detector,...) .
136
137         Compatible with many editors and the existing golden/slicer schemas
138         (extra columns are harmless).

```

```

139     """
140     segs = _to_list(segments)
141     lines: List[str] = []
142     if include_header:
143
144     lines.append("start_time,end_time,duration,shot_count,shot_count_confidence,detector,id")
145     for s in segs:
146         md = s.get("metadata") or {}
147         shot = md.get("shot_count", "")
148         conf = md.get("shot_count_confidence", "")
149         det = md.get("detector", "")
150         lines.append(
151             f"{s.get('start_time',0)},{s.get('end_time',0)},{s.get('duration',0)},"
152             f"{shot},{conf},{det},{s.get('id','')}"
153         )
154     return "\n".join(lines) + "\n"
155
156 def segments_to_edl(segments: Iterable[Dict[str, Any]], title: str = "Highlights") ->
str:
157     """Return a simple EDL (CMX 3600-ish) for the segments.
158
159     Many NLEs will ingest this for a rough cut. Uses record timecode 00:00:00:00

```

4. assistant (2026-06-25T17:38:33.404Z)

Now let me check the worker `intervals.csv` reference and the `\_segment\_metadata` helper to confirm the contrast claims.

5. user (2026-06-25T17:38:34.632Z)

```

1     """Minimal Value Layer exports (first slice of review item 3).
2
3     Provides timeline exports (EDL, JSON, CSV) and basic stats over the
4     play_segments returned by the indexer (or query_play_segments). These are
5     the "time-saved editing is table stakes" deliverables and also give a
6     machine-readable foundation for later human correction / golden feeding
7     and clip galleries.
8
9     Kept deliberately small and pure (no DB, no side effects) so they are
10    easy to call from CLI, notebooks, or a future API endpoint.
11    """
12
13    from __future__ import annotations
14
15    import json
16    from typing import Any, Dict, Iterable, List, Optional
17
18
19    def _to_list(segments: Iterable[Dict[str, Any]]) -> List[Dict[str, Any]]:
20        return list(segments) if not isinstance(segments, list) else segments
21
22
23    def _segment_metadata(s: Dict[str, Any]) -> Dict[str, Any]:
24        """A play segment's metadata as a dict, whether stored inline (segmenter output)
25        or as a JSON string (DB round-trip via query_play_segments)."""
26        md = s.get("metadata") or {}
27        if isinstance(md, str):
28            try:
29                md = json.loads(md)
30            except Exception:
31                md = {}
32        return md if isinstance(md, dict) else {}
33
34

```

```

35     def _fmt_hms(seconds: float) -> str:
36         """HH:MM:SS, matching the segmenters' _fmt_ts so the summary block lines up with
37         the per-rally [rally] log lines and the existing timestamp style."""
38         total = max(0, int(float(seconds)))
39         h, rem = divmod(total, 3600)
40         m, s = divmod(rem, 60)
41         return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     def format_rally_summary(
45         segments: Iterable[Dict[str, Any]], title: str = "Rally summary"
46     ) -> str:
47         """Render a compact, scannable end-of-run block for manual QA against the rally CSV.
48
49         One header line (total rallies + total highlight duration in HH:MM:SS) followed by a
50         per-rally table ``# start end dur(s) shot_count`` in time order. Pure (no DB, no
I/O):
51         the same play_segment dicts that feed ``segments_to_csv`` / the rally CSV go in and
52         text comes out, so the log and CSV line up row-for-row for eyeball verification.
53
54         A segment with no ``shot_count`` in its metadata renders the count as ``-``.
55         """
56         segs = _to_list(segments)
57         total_dur = sum(
58             float(s.get("duration", float(s.get("end_time", 0)) - float(s.get("start_time",
59             0))))
59             for s in segs

```

6. assistant (2026-06-25T17:38:37.033Z)

Now let me confirm the claim that the `shot\_count` key is now present with value `None` for motion segments. Let me look at the PR diff and the motion segmenter.

7. user (2026-06-25T17:38:38.589Z)

```

eabf774 fix(B1/P9): remove fabricated data from motion segmenter
cd5b3d2 feat(01/P11): Gemini remote-file GC ? startup orphan sweep (#380)
86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)
71a8241 chore(repo): fix corrupted .gitignore (UTF-16 tail) + ignore artifacts/, track product
overview (#377)
---DIFF---
backend/main.py | 26 ++++++
backend/pipeline/segmenters/motion.py | 100 ++++++-----
backend/providers/gemini.py | 70 ++++++-----
docs/CODE_CLEANUP_AUDIT_2026-06-24.md | 4 +-
tests/test_motion_segmenter.py | 46 +----
tests/test_provider_gemini.py | 135 ++++++-----
6 files changed, 269 insertions(+), 112 deletions(-)

```

8. user (2026-06-25T17:38:41.226Z)

```

diff --git a/backend/pipeline/segmenters/motion.py b/backend/pipeline/segmenters/motion.py
index 9b714c5..36d6b36 100644
--- a/backend/pipeline/segmenters/motion.py
+++ b/backend/pipeline/segmenters/motion.py
@@ -1,5 +1,4 @@
 import logging
 -import random
 import time
 from typing import Any, Dict, List, Optional, Tuple

@@ -22,12 +21,11 @@ class MotionPlaySegmenter(BasePlaySegmenter):

```

```

def process_video(self, video_id: str, video_path: str, player_pool: Optional[List[str]] =
None, max_frames: Optional[int] = None) -> "Result[List[Dict[str, Any]]]":
    """
    - Processes a video to detect play segments using motion heuristics,
    - populates SQLite with detected segments, and indexes detailed events.
    + Processes a video to detect play segments using motion heuristics and
    + populates SQLite with detected segments. The ``player_pool`` arg is
    + accepted for ABC signature compatibility but no longer used (the motion
    + segmenter has no ground truth for server/receiver ? see _populate_db).
    """
    - if not player_pool:
    -     player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
    -
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        logger.error(f"Segmenter could not open video: {video_path}")
@@ -35,7 +33,7 @@ class MotionPlaySegmenter(BasePlaySegmenter):

    final_segments = self._detect_motion_segments(cap, max_frames)

    - segments_data = self._populate_db_with_events(video_id, final_segments, player_pool)
    + segments_data = self._populate_db(video_id, final_segments)

    return Success(value=segments_data)

@@ -184,81 +182,35 @@ class MotionPlaySegmenter(BasePlaySegmenter):

    return final_segments

- def _populate_db_with_events(self, video_id: str, final_segments: List[Tuple[float,
float]],
-                             player_pool: List[str]) -> List[Dict[str, Any]]:
-     """Persist ``final_segments`` and their fabricated metadata/events to the DB.
+ def _populate_db(self, video_id: str, final_segments: List[Tuple[float, float]]
+ ) -> List[Dict[str, Any]]:
+     """Persist ``final_segments`` to the DB with honest metadata.

    - For each detected segment this assigns a random server/receiver, fabricates
    - rich tags, then indexes a serve / 0?3 smash / termination event set ? all
    - with the stdlib ``random`` module (the legacy demo's fabricated baseline).
    - Returns the per-segment dicts handed back to the caller. The ``random``
    - call order here is load-bearing for reproducibility and must not change.
    + The motion segmenter detects play segments via pixel-diff heuristics but
    + has NO ground truth for who served, shot counts, shuttle speed, or event
    + breakdown ? those were previously fabricated with the stdlib ``random``
    + module (the legacy demo baseline). They are now persisted as ``None`` /
    + unknown markers so consumers can distinguish "not modeled" from a real
    + value. Matches yolo_hybrid/gemini, which pass ``server=None, receiver=None``.
    """
    - # Populate SQLite DB with segments and metadata events
    segments_data = []
    for start, end in final_segments:
        - # Assign Server and Receiver randomly from pool
        - server, receiver = random.sample(player_pool, 2)
        -
        - # Formulate dynamic rich tags
        dur = end - start
        metadata = {
            - "shot_count": random.randint(4, 25),
            + "shot_count": None,
            - "intensity": "high" if dur > 12 else "medium",
            - "avg_speed_kmh": round(random.uniform(40, 85), 1)
            + "avg_speed_kmh": None,
        }
    -

```

```

-         segment_id = self.db.add_play_segment(video_id, start, end, server, receiver,
metadata)
-
-         if segment_id:
-             # Add sub-segment events
-             # A: Serve Event (always at start)
-             serve_foul = random.random() < 0.15 # 15% chance of serve foul
-             self.db.add_event(
-                 segment_id=segment_id,
-                 timestamp=start + 0.5,
-                 event_type="serve",
-                 player=server,
-                 details={"status": "foul" if serve_foul else "valid", "type": "low_serve"
if random.random() > 0.5 else "high_serve"}
-             )
-
-             # B: Smash Events (1 to 3 random smashes per segment)
-             smashes_count = random.randint(0, 3)
-             for _ in range(smashes_count):
-                 smash_time = random.uniform(start + 1.5, end - 1.0)
-                 smasher = random.choice([server, receiver])
-                 self.db.add_event(
-                     segment_id=segment_id,
-                     timestamp=round(smash_time, 2),
-                     event_type="smash",
-                     player=smasher,
-                     details={"speed_kmh": round(random.uniform(120, 250), 1)}
-                 )
-
-             # C: Segment Termination Event (foul/winner)
-             foul_types = ["out_of_bounds", "net_touch", "double_hit", "fault"]
-             is_foul = random.random() > 0.4
-             error_player = random.choice([server, receiver])
-
-             self.db.add_event(
-                 segment_id=segment_id,
-                 timestamp=end - 0.2,
-                 event_type="foul" if is_foul else "winner",
-                 player=error_player,
-                 details={"type": random.choice(foul_types)} if is_foul else {"shot_type":
"drop" if random.random() > 0.5 else "smash"}
-             )
-
-             # Store segment representation to return to caller
-             segments_data.append({
-                 "id": segment_id,
-                 "start_time": start,
-                 "end_time": end,

```

9. assistant (2026-06-25T17:38:44.027Z)

The diff confirms the core premise: `shot\_count` is now explicitly set to `None` in the metadata dict (previously `random.randint(4, 25)`). Let me see the rest of the new `\_populate\_db` to confirm the metadata dict is attached to the returned segment dicts.

10. user (2026-06-25T17:38:44.846Z)

```

-         "end_time": end,
-         "duration": dur,
-         "server": server,
-         "receiver": receiver,
-         "metadata": metadata
-     })
-
+     segment_id = self.db.add_play_segment(video_id, start, end, None, None, metadata)

```

```
+         segments_data.append({
+             "id": segment_id,
+             "start_time": start,
+             "end_time": end,
+             "duration": dur,
+             "server": None,
+             "receiver": None,
+             "metadata": metadata,
+         })
+     return segments_data
```

```
# Register the segmenter
```

11. assistant (2026-06-25T17:38:48.496Z)

Confirmed. The motion segments now carry `metadata = {"shot\_count": None, ...}`. So in `segments\_to\_csv`, `md.get("shot\_count", "")` returns `None` (key present), and the f-string `f"...{shot}..."` renders `None` as the literal string `None`.

Now let me verify the worker `intervals.csv` claim ? that it reads a flat dict key absent for motion (so unaffected).

12. user (2026-06-25T17:38:49.143Z)

```
70         setattr(obj, parts[-1], _coerce(val))
71
72
73     def _write_intervals_csv(segments: List[dict], path: str) -> None:
74         """Decimal-second `[start,end)` rallies + shot count / ending reason ? the join-
friendly
75         artifact (same schema as the rally-annotator labels)."""
76         with open(path, "w", newline="") as f:
77             w = csv.writer(f)
78             w.writerow(["start", "end", "shot_count", "ending_reason"])
79             for s in segments:
80                 w.writerow([
81                     f'{float(s.get("start_time", 0.0)):.3f}',
82                     f'{float(s.get("end_time", 0.0)):.3f}',
83                     s.get("shot_count", ""),
84                     s.get("ending_reason", s.get("shot_count_confidence", "")),
85                 ])
86
87
88     def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) ->
None:
89         with open(path, "w") as f:
90             json.dump({
91                 "video_id": video_id,
92                 "status": status,
93                 "segment_count": len(segments),
94                 "segments": [{"start": float(s.get("start_time", 0.0)),
```

13. assistant (2026-06-25T17:38:54.438Z)

The worker's `\_write\_intervals\_csv` reads `s.get("shot\_count", "")` ? a flat top-level key. The motion segment dicts (from `\_populate\_db`) have `shot\_count` nested inside `metadata`, NOT at the top level. So for motion segments the top-level key is absent, the `""` default applies, and the cell renders blank. This confirms the finding's claim that the worker `intervals.csv` is unaffected.

Let me verify the finding's claim that `format\_rally\_summary` (the rally-summary table) correctly handles `None` ? which I already saw at lines 73-74: `shot = \_segment\_metadata(s).get("shot\_count")` then `shot\_str = "-" if shot is None else str(shot)`. That correctly renders `-` for `None`. Confirmed.

Now let me double-check there isn't a test that asserts the CSV produces empty for motion (which would mean it's already handled or that the finding is wrong about behavior).

14. user (2026-06-25T17:38:55.863Z)

```
tests\test_api_cloud_dispatch.py:60: w.writerow(["start", "end", "shot_count",
"ending_reason"])
tests\test_api_cloud_dispatch.py:121: assert {md.get("shot_count") for md in metas} == {5, 3}
tests\test_api_endpoints.py:344: sid = db.add_play_segment("bvid", 1.0, 4.0) # no shot_count
? exempt from the floor
tests\test_api_endpoints.py:449: def test_compile_enforces_config_min_shot_count(env):
tests\test_api_endpoints.py:450: """stitching.min_shot_count applies on the API path (parity
with
tests\test_api_endpoints.py:452: WITHOUT shot_count metadata are exempt (explicitly requested
ids must not
tests\test_api_endpoints.py:455: m.global_config.stitching.min_shot_count = 8
tests\test_api_endpoints.py:457: s_low = db.add_play_segment("vid", 1.0, 4.0, None, None,
{"shot_count": 5})
tests\test_api_endpoints.py:458: s_high = db.add_play_segment("vid", 10.0, 14.0, None, None,
{"shot_count": 12})
tests\test_api_endpoints.py:459: s_unknown = db.add_play_segment("vid", 20.0, 24.0) # no
shot_count metadata
tests\test_api_endpoints.py:483: def test_compile_400_when_all_below_min_shot_count(env):
tests\test_api_endpoints.py:485: m.global_config.stitching.min_shot_count = 8
tests\test_api_endpoints.py:487: sid = db.add_play_segment("vid", 1.0, 4.0, None, None,
{"shot_count": 3})
tests\test_api_endpoints.py:491: assert "min_shot_count" in r.json()["detail"]
tests\test_gemini.py:50: assert res[0]["metadata"]["shot_count"] == 5
tests\test_motion_segmenter.py:18: fabricated server/receiver/shot_count/events are
reproducible. The random call
tests\test_motion_segmenter.py:149: server/receiver/shot_count/speed/events). Boundaries
still come from the motion loop."""
tests\test_motion_segmenter.py:193: "metadata": {"shot_count": None, "intensity": "medium",
"avg_speed_kmh": None},
tests\test_motion_segmenter.py:198: {"shot_count": None, "intensity": "medium",
"avg_speed_kmh": None}),
tests\test_rally_slicer.py:6: segments_to_csv,
tests\test_rally_slicer.py:88: {"id": 1, "start_time": 10.0, "end_time": 15.0,
"duration": 5.0, "metadata": {"shot_count": 4, "detector": "wasb-hybrid-gemini"}},
tests\test_rally_slicer.py:89: {"id": 2, "start_time": 30.0, "end_time": 33.5,
"duration": 3.5, "metadata": {"shot_count": 3, "detector": "wasb-hybrid-gemini"}},
tests\test_rally_slicer.py:108: def test_segments_to_csv_has_header_and_data():
tests\test_rally_slicer.py:109: csv = segments_to_csv(_segs())
tests\test_rally_summary.py:10: def _seg(start, end, shot_count=None, metadata=None):
tests\test_rally_summary.py:14: elif shot_count is not None:
tests\test_rally_summary.py:15: seg["metadata"] = {"shot_count": shot_count}
tests\test_rally_summary.py:28: def test_per_rally_rows_use_hms_and_shot_count():
tests\test_rally_summary.py:37: def test_missing_shot_count_renders_dash():
tests\test_rally_summary.py:43: # DB round-trip stores metadata as a JSON string; the summary
must still read shot_count.
tests\test_rally_summary.py:44: seg = _seg(0, 5, metadata='{"shot_count": 7}')
tests\test_tracknet_hybrid.py:194: assert meta["shot_count_confidence"] == "LocalNoAI"
tests\test_trajectory_hybrid_equivalence.py:4: shot_count parse); both are now thin subclasses of
TrajectoryHybridSegmenter.
tests\test_trajectory_hybrid_equivalence.py:78: def test_shot_count_behavior_is_identical(cls,
family, detector, exchange, expected):
tests\test_trajectory_hybrid_equivalence.py:83: assert res[0]["metadata"]["shot_count"] ==
expected
tests\test_worker.py:39: segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
tests\test_worker.py:44: assert lines[0] == "start,end,shot_count,ending_reason"
tests\test_worker.py:87: {"start_time": 2.0, "end_time": 5.0, "shot_count": 4,
"shot_count_confidence": "LocalNoAI"},
tests\test_worker.py:88: {"start_time": 9.0, "end_time": 12.0, "shot_count": 6,
"shot_count_confidence": "LocalNoAI"},
```


15. assistant (2026-06-25T17:38:59.105Z)

The CSV tests at `test\_rally\_slicer.py` use segments with real integer `shot\_count` values (4, 3), not `None`, so they don't exercise the `None`-rendering path. The finding is not contradicted by any existing test. Let me confirm Python's f-string behavior for `None` to be fully rigorous.

16. user (2026-06-25T17:39:00.201Z)

a, None, b
None

17. assistant (2026-06-25T17:39:09.244Z)

Behavior confirmed empirically: with `metadata = {"shot\_count": None}`, `md.get("shot\_count", "")` returns `None` (default not applied since key present), and the f-string interpolation produces the literal string `None` in the CSV cell.

All claims in the finding verified against the actual code:

- `segments\_to\_csv` (export.py:146) does `shot = md.get("shot\_count", "")` and interpolates it directly into the f-string at line 151.
- The PR sets motion segment metadata `shot\_count` to `None` (key now present), so the `""` default never applies ? CSV renders literal `None`.
- The contrast is accurate: `format\_rally\_summary` (export.py:73-74) correctly guards with `"-` if shot is None else str(shot)`.
- The worker `intervals.csv` (worker/___main___py:83) reads a flat top-level `shot\_count` key that is absent for motion segments (their `shot\_count` is nested under `metadata`), so the `""` default applies and it is unaffected.
- No existing test exercises the `None` CSV path, so it's not already handled.

This is a real but genuinely cosmetic inconsistency. The CSV will show `None` instead of blank/`-` for motion-segmenter rows. No crash, no data corruption, no consumer-breaking behavior described. It's an inconsistency with the sibling rendering convention. NIT severity is appropriate (the reviewer self-classified it as such and called it "optional polish, not blocking").

18. user (2026-06-25T17:39:15.064Z)

Structured output provided successfully

19. assistant (2026-06-25T17:39:20.270Z)

Verified. The finding is real and correctly classified as a NIT.

The motion segmenter in PR #385 now sets `metadata = {"shot\_count": None, ...}`, so in `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\tools\export.py:146` the call `md.get("shot\_count", "")` returns `None` (key present, default skipped), and the f-string at line 151 interpolates it as the literal string `None` in the CSV cell. The contrast claims hold: `format\_rally\_summary` (export.py:73-74) correctly guards `None` with `"-`, and the worker's `intervals.csv` (worker/___main___py:83) reads a flat top-level key that motion segments don't have, so it's unaffected. Purely cosmetic, non-blocking.